A Deep Learning Pipeline For Human Activity Recognition

Bojan Dimovski
4th year student
Faculty of Electrical Engineering and Information Technologies
Skopje, Macedonia
bojandimovski9@gmail.com

Ana Indovska
4th year student
Faculty of Electrical Engineering and Information Technologies
Skopje, Macedonia
kti792022@feit.ukim.edu.mk

Marija Boshkoska
4th year student
Faculty of Electrical Engineering and Information Technologies
Skopje, Macedonia
kti1212022@feit.ukim.edu.mk

Tomi Nikoloski
4th year student
Faculty of Electrical Engineering and Information Technologies
Skopje, Macedonia
kti792022@feit.ukim.edu.mk

***Abstract*—Human Activity Recognition (HAR) using wearable sensors often struggles to differentiate between highly similar static behaviors, such as sitting and standing. This study presents a robust HAR framework utilizing IMU and pressure sensor data collected over a one-month period. We implement a custom CNN1D-LSTM architecture equipped with a novel Absolute Max (ABSMAX) pooling layer designed to eliminate signal sign bias. Initial 6-fold cross-validation yielded exceptional performance by grouping static behaviors into a single "upright still" class (macro F_1-score: 0.996), demonstrating strong generalization on an external validation dataset. However, directly training the network to separate sitting and standing labels proved unsuccessful, degrading to a sitting F_1-score of 0.536 due to overlapping sensor profiles. To overcome this limitation, we introduce a state-transition modeling approach via heuristic post-processing. The core model was re-engineered to identify an aggregate "upright still" state alongside explicit dynamic "transition up" and "transition down" events, improving sitting and standing F_1-scores to 0.848 and 0.923, respectively. Performance was optimized further by utilizing active dynamic behaviors (walking, running, and stair locomotion) as reliable anchor states to deterministically reset and ground the contextual tracking loop. This final pipeline achieved a prominent macro F_1-score of 0.977 across all granular activities. Analysis indicates that the remaining bottleneck resides in sitting recall (0.845) due to subtle transition-down misclassifications. Future work will focus on optimizing model sensitivity to brief postural transitions to achieve flawless static state tracking.**

***Index Terms*—Human Activity Recognition (HAR), Deep learning, Sensor fusion, Inertial Measurement Unit (IMU), Pressure sensors, CNN-LSTM architecture, Absolute Max (ABSMAX) pooling, Post-processing, State-transition modeling, Postural transitions, Anchor states**

I. Introduction

Human Activity Recognition (HAR) frameworks typically treat time-series sensor windows as isolated targets. While this works for dynamic locomotion, it fails on structurally similar static postures like sitting and standing. Because body-worn IMUs record heavily overlapping feature distributions during motionless states, naive networks trained on independent sitting and standing labels suffer from severe cross-contamination. This orientation and signal bottleneck is well-documented; as shown by [1], a single torso-worn accelerometer exhibits virtually identical spatial configurations during both postures, causing standard machine learning algorithms to break down and yield an accuracy of just 62% on these specific states. To resolve this, [1] shifted the paradigm toward context-aware modeling by introducing a Multiple Contexts Ensemble (MCE) that leverages past activity history and dynamic transitions to sequentially separate the states via majority voting.

In this work, we modernize and scale this context-driven philosophy. Instead of hand-crafting eleven separate algorithmic and statistical contexts for an ensemble, we offload deep spatial-temporal pattern extraction to a unified 1D-CNN-LSTM core equipped with a custom Absolute Max pooling layer… For example, the multimodal architecture by [2] encountered massive confusion matrix overlap between these states, illustrating the limits of purely instantaneous pattern recognition.

To solve this, we shift the burden of static state tracking from neural pattern recognition to temporal context modeling. At the feature level, we use a 1D-CNN-LSTM network with a custom Absolute Max (ABSMAX) pooling layer. While traditional Max Pooling discards negative peak deviations, our ABSMAX layer captures absolute bidirectional physical extremes from transducer and accelerometer streams, yielding a consistent performance edge.

The primary breakthrough, however, is our training and post-processing topology. Instead of forcing the network to directly separate ambiguous static signals—which yields a poor sitting F_1-score of 0.536—we aggregate both behaviors into a single "upright_still" class during neural training. Concurrently, the network is trained to isolate brief, dynamic postural boundaries: "transition_up" and "transition_down" triggers.

We then leverage a heuristic post-processing loop that tracks these transitional triggers alongside active "anchor states" (walking, running, stair locomotion) to sequentially reconstruct the static states. By treating locomotion as a hard reset for tracking logic and using transitions to flip the behavioral state, our pipeline successfully recovers granular static classifications, driving the final macro F_1-score to 0.977.

The document is organized as follows: Section 2 covers preprocessing: IMU vs. pressure sensor; Section 3 details the deep learning architecture (including the ABSMAX layer and loss functions); Section 4 compiles results and provides the discussion; and Section 5 outlines future work.

II. Preprocessing

A. Preprocessing of IMU signals

The physical raw telemetry from the body-worn Inertial Measurement Unit (IMU) is highly susceptible to high-frequency motion artifacts, structural sensor jitter, and gravity-induced signal drifts. To extract a clean kinematic feature space, the raw three-axis accelerometer channels (`Accelerometer/Raw.X`, `Y`, `Z`) and three-axis gyroscope channels (`Gyroscope/Raw.X`, `Y`, `Z`) undergo localized normalization and filtering.

Outliers are isolated using a thresholding approach, and signal noise is mitigated by applying a low-pass Butterworth filter configured with a fifth-order cutoff at $f_{\text{high\_cutoff}} = 4$ Hz based on the system sampling frequency of 50 Hz. This filter attenuates high-frequency sensor noise while preserving the low-frequency structural components corresponding to physical body movements. Following filtering, a smoothing rolling window is applied to stabilize steady-state acceleration trajectories. To maintain invariant spatial representations regardless of minor shifts in physical sensor orientation, the total acceleration magnitude (`Accelerometer/Magnitude`) and angular velocity magnitude (`Gyroscope/Magnitude`) are explicitly derived via Euclidean norms and appended to the model's feature stream.

B. Preprocessing of the Pressure sensor signal

Unlike inertial signals that map immediate spatial acceleration, the absolute raw pressure sensor stream (`Pressure/Raw`) captures the magnitude of localized physical forces during postural contact and ground impact. To make this signal useful for activity boundaries, a specialized preprocessing pipeline is executed.

First, the absolute pressure signal is smoothed using a rolling mean filter to filter out micro-oscillations caused by shifting body weight during continuous static poses. The primary breakthrough in our pressure feature extraction, however, is the computation of its first-order discrete derivative, which is outputted as `Pressure/Raw.Deriv`.

Absolute pressure baselines fluctuate drastically between different users due to varying weights or baseline sensor fitting snugness. Furthermore, absolute pressure metrics are inherently sensitive to external environmental factors, meaning baseline readings vary heavily depending on the altitude of the recording location, atmospheric pressure conditions, room humidity, and ambient temperature. The derivative bypasses these issues by isolating the **rate of change** of force. During prolonged static states (sitting or standing), the pressure derivative stabilizes tightly near zero. Conversely, during a postural shift, the derivative exhibits a pronounced, sharp bipolar spike, surging aggressively during a down-transition (compression) and dropping sharply during an up-transition (decompression). This mathematical transformation normalizes baseline offsets between subjects and environments, explicitly exposing state-switching boundaries to the downstream neural network.

C. Segmentation and Window Labeling Topology

The preprocessed continuous time-series data streams are sliced into discrete temporal windows to serve as inputs for the neural network. Both modeling pipelines configure a window size (W) of 4 seconds (200 samples at 50 Hz) and a sliding step (S) of 1 second (50 samples), yielding a 75% overlap between consecutive frames. However, they enforce fundamentally different labeling constraints:

1) **Naive 5-Class Segmentation Strategy:** In this baseline setup, window labels are assigned deterministically based on absolute ground-truth segments across the targeted postures. A window is only retained for training or validation if it is entirely homogeneous, meaning every single sample within the 4-second span shares the exact same activity label (e.g., pure `walking`, `running`, `laying`, `sitting`, or `standing`). If a window intersects any boundary where an activity switches, it is discarded to prevent feeding corrupted, mixed-signal profiles into the network.
2) **Transition-Aware Model Strategy:** Our optimized framework re-engineers window labeling to explicitly model temporal boundaries over the targeted behavioral categories (`upright_still`, `walking`, `running`, `laying`, `stairs_up`, `stairs_down`, `transition_up`, `transition_down`).
   - **Static/Dynamic Class Windows:** For non-transition classes like `walking`, `running`, or the merged `upright_still`, the window labeling logic requires full homogeneity. Every single sample within the window must belong to that specific continuous behavior to represent a completely stable state.
   - **Transition Class Windows:** A transition window is intentionally constructed over the boundary zones where activities switch. Instead of requiring homogeneity, a window is explicitly labeled as `transition_up` or `transition_down` if it encompasses the exact interval where a user transitions between sitting and standing. These windows capture the non-stationary, transient sensor signatures, such as the rapid spike in `Pressure/Raw.Deriv` paired with low-frequency IMU shifts, enabling the neural core to predict state boundaries before the post-processing loop sequentially tracks them.

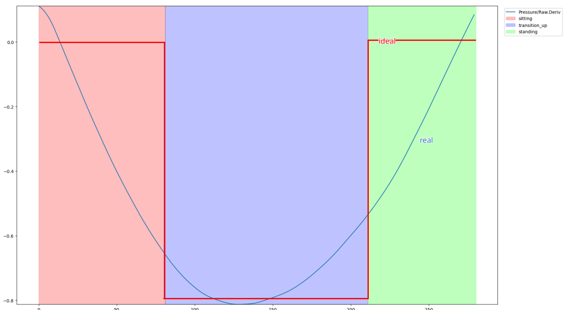

Fig. 1. Pressure deriv. for window labeled as `transition_up`

Fig. 1 shows a graph of how the derivative of the pressure sensor signal (the final processed signal ready to be used as a input to one of the channels of the model) looks. As can be seen, a window that's labeled as `transition_up` is a segment that begins with `sitting`, the person stands up and ends with `standing`, so as to capture the entire 4 seconds. In the `sitting` and `standing` subsegments, this signal has an amplitude of around 0, which is expected since the pressure is constant for static activities, whereas in the `transition_up` segment, the derivative $\frac{dp(t)}{dt}$ takes a dip and takes on negative values, because the pressure decreases during that tiny timeframe.

III. Model architecture

A. General architectural overview

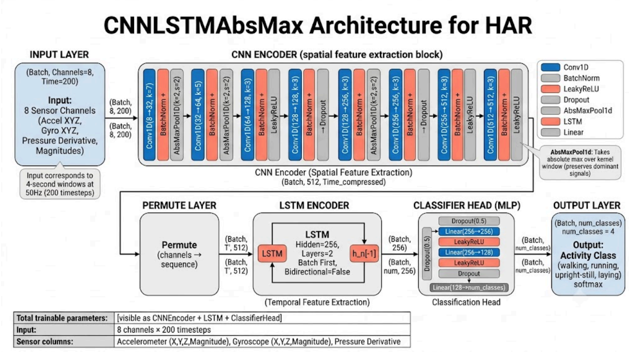

Fig. 2. Flow diagram of the architecture of the CNN1D-LSTM DL model

The neural network core maps segmented multi-channel time-series inputs into deep spatial-temporal representations using a unified 1D-CNN-LSTM framework. Slicing raw window streams directly into temporal sequences exposes high-frequency sensor relationships to consecutive processing stages.

The front-end feature extractor, or encoder, consists of three sequential one-dimensional convolutional blocks (`ConvBlock`). Each individual block implements a 1D convolutional layer (`nn.Conv1d`) designed to automatically synthesize local localized patterns along the temporal axis. This configuration utilizes a weight initialization technique (`kaiming_normal_`) matched to the activation topology, paired with a kernel size of 5 and zero-padding to sustain stable boundary dimensions. Every convolutional layer is directly coupled with a 1D Batch Normalization layer (`nn.BatchNorm1d`) to stabilize training dynamics, enforce regularized internal covariate distributions, and accelerate optimization convergence.

To introduce non-linearity, the network applies a Leaky Rectified Linear Unit (Leaky ReLU) function configured with a negative slope of 0.01 instead of a standard ReLU. While standard ReLU completely zeroes out negative values, creating a "dying ReLU" vulnerability where neurons become permanently inactive during backpropagation, Leaky ReLU maintains a small, non-zero gradient for negative inputs. This property ensures continuous weight updates across all physical channels, which is highly beneficial for oscillatory sensor streams like accelerometer and gyroscope coordinates that fluctuate across zero.

Following structural downsampling via pooling layers, the spatial feature representations are reorganized by permuting their channel dimensions to match sequential requirements. These deep representations are fed into a recurrent neural network block consisting of a single-layer Long Short-Term Memory (LSTM) network with a hidden dimension size of 128. This recurrent architecture uses orthogonal weight initializations to model systemic dependencies across consecutive intervals. The terminal hidden state vector (h_n) from the LSTM sequence is extracted to provide a dense, context-aware summary of the entire time window. This summary passes through a specialized multi-layer classifier head containing linear projections, Batch Normalization steps, and dropout regularizations to yield final class probability scores.

B. Max-Pooling vs. Absmax-Pooling

Standard deep learning pipelines traditionally use Max Pooling layers to shrink temporal dimensions and enforce local translational invariance. However, conventional maximum extraction exhibits a fundamental signal sign bias when mapping zero-centered physical signals like inertial raw telemetry. In standard configurations, a regular Max Pooling layer selects the most positive scalar value within its kernel window. If a specific time series span contains a massive negative peak acceleration (e.g., $-1.5g$) alongside a weak positive deviation (e.g., $+0.2g$), the standard operator discards the physical extreme and registers the minor positive value. This operation effectively ignores crucial structural spikes that capture abrupt physical impacts or fast positional shifts.

To eliminate this asymmetry, we designed a custom Absolute Max Pooling layer (`AbsMaxPool1d`). Our custom layer unfolds the temporal feature tensors, identifies the indices where the absolute magnitude is maximized, and uses those indexes to extract the raw bidirectional extreme values. This architecture retains the original signal signs while capturing the true physical peaks, as can be seen in Fig. 3.

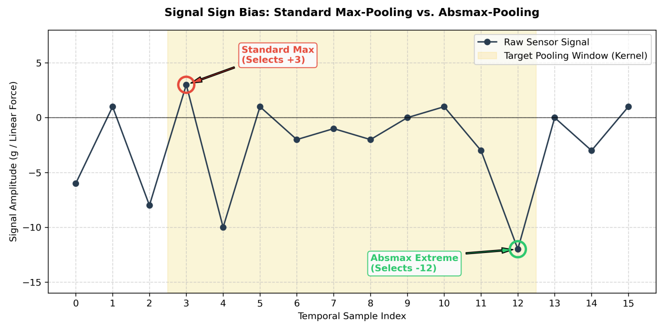

Fig. 3. Visualization of signal sign bias within a target downsampling kernel window.

Empirical evaluation reveals that although the macro-level differences appear marginal, the custom absolute pooling configuration consistently outperforms traditional maximum selection across both local validations and external validation sets:

- **6-Fold Cross-Validation Macro F_1-Score:** Standard Max Pooling achieves a macro F_1 metric of 0.99531, while our custom ABSMAX layer improves the validation metric to 0.99568.
- **External Dataset (`HAR_jumping`) Macro F_1-Score:** Standard Max Pooling yields a validation score of 0.99314, whereas the ABSMAX topology drives performance up to 0.99674.

While these performance gains emerge in the third and fourth decimal places, they consistently demonstrate that preserving bidirectional physical signal extremes enhances the model robustness and generalization capacity across unseen tracking data.

C. Loss function and Optimization

Model parameter optimization is managed by a structured training pipeline that runs for a fixed duration of 200 epochs. The training process uses a multi-class Cross-Entropy Loss function (`nn.CrossEntropyLoss`) as its objective optimization criterion, which penalizes divergence between predicted categorical distributions and true activity classes.

The gradient descent path is driven by the AdamW optimizer, using an initial learning rate of 10^{-3} combined with a weight decay regularizer of 10^{-4} to mitigate overfitting tendencies. To prevent optimization plateaus, the learning rate updates continuously across epochs using a Cosine Annealing learning rate scheduler [3] (`CosineAnnealingLR`). This scheduler smoothly decays the step size down to a minimum target threshold of 10^{-6} using a cosine curve, allowing the model to make rapid initial adjustments before executing stable, high-precision weight tuning during late-stage training cycles.

IV. Results and Empirical Evaluation

A. Max-Pooling vs. Absmax-Pooling: 6-Fold Cross-Validation

To establish an experimental baseline, the network was evaluated using a 6-Fold Cross-Validation (6FCV) scheme. This stage groups the ambiguous static postures into a single aggregate `upright_still` class alongside the dynamic activities.

a) *Standard Max-Pooling Baseline:* When employing conventional 1D Max-Pooling, the model struggles slightly with bidirectional physical sensor extremes due to positive scalar extraction bias. The confusion matrix for this configuration is illustrated in Fig. 4.

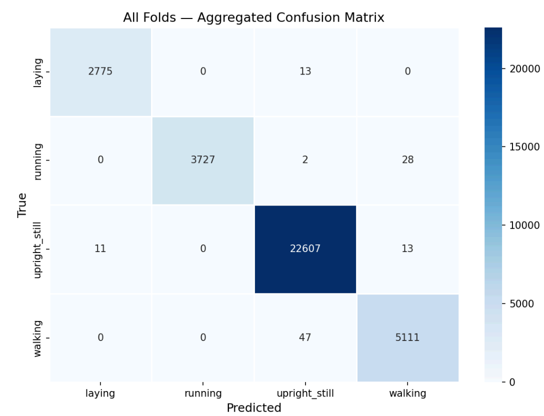

Fig. 4. Confusion matrix for the 6-Fold Cross-Validation baseline utilizing standard Max-Pooling.

This approach yields a macro F_1-score of 0.9953. While highly accurate overall, specific structural leakage occurs between motionless segments and the `laying` posture due to discarded peak acceleration sign features.

b) *Custom Absmax-Pooling Performance:* Replacing the standard pooling layer with our custom `AbsMaxPool1d` architecture eliminates sign bias by selecting the absolute highest physical signal intensities. This modification improves the cross-validation macro F_1-score to 0.9956.

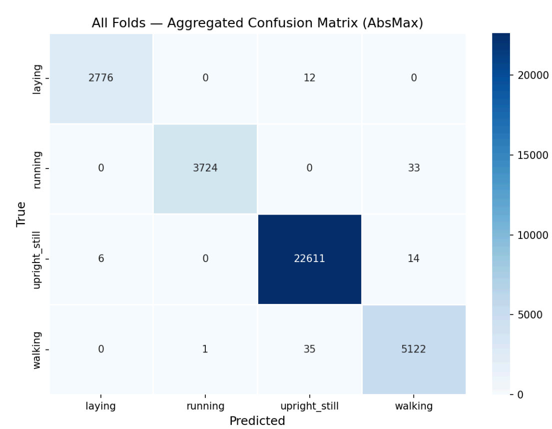

Fig. 5. Confusion matrix for the 6-Fold Cross-Validation utilizing Absmax-Pooling.

As shown in Fig. 5, extracting true bidirectional amplitude responses reduces systemic classification anomalies. The errors that do occur shift toward highly logical boundaries —such as a single isolated transition between `walking` and running—while reducing severe structural misclassifications between standard motionless activities and laying.

B. Generalization on External Test Datasets (HAR_jumping)

To confirm model robustness outside the primary cross-validation domain, both pooling methods were tested against the completely independent HAR_jumping external dataset.

a) *Max-Pooling Evaluation:* Under external validation conditions, the traditional Max-Pooling layer registers a macro F_1-score of 0.9931. The corresponding evaluation profile is displayed in Fig. 6.

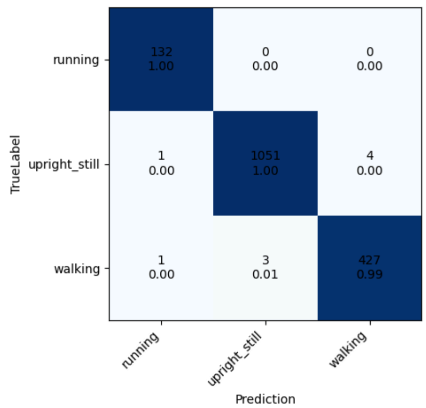

Fig. 6. External dataset confusion matrix utilizing a standard Max-Pooling architecture.

b) *Absmax-Pooling Evaluation:* The absolute maximum pooling layer handles the unseen sensor artifacts of the external test set effectively, lifting the evaluation metric to a macro F_1-score of 0.9967.

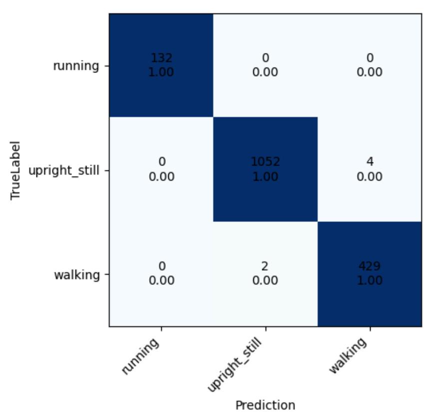

Fig. 7. External dataset confusion matrix utilizing our custom Absmax-Pooling architecture.

As shown in Fig. 7, capturing absolute physical peaks maintains consistent model performance. It filters out sensor drift differences between separate validation populations, proving more resilient than traditional maximum reduction setups.

C. Naive Classification: The Challenge of Sitting-Standing Separation

While the merged upright_still topology yields excellent accuracy metrics, splitting sitting and standing back into independent targets using a standard classification framework proves highly challenging. This experiment was executed using only the high-performing Absmax-pooling framework.

As can be seen, directly classifying these states degrades performance significantly. The static posture similarity causes massive feature cross-contamination, dropping the sitting class F_1-score to a poor 0.535, with a true positive recall rate of only 0.470. Because the instantaneous IMU footprints match almost exactly, the network misclassifies nearly half of the continuous sitting intervals as standing.

D. State-Transition Modeling and Heuristic Post-Processing

To resolve this limitation without sacrificing granular activities, the system shifts tracking responsibilities to a state-transition framework. The model is trained on a unified upright_still class along with two dynamic boundary events: transition_up and transition_down.

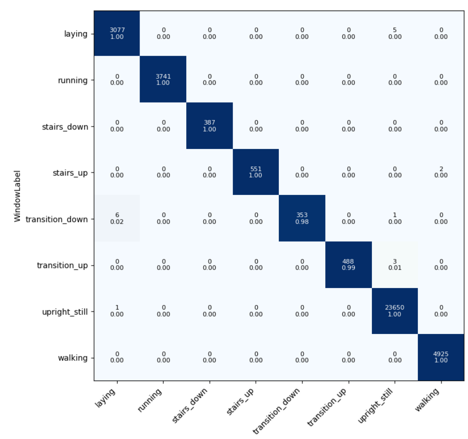

Fig. 8. Raw multi-class model output matrix including explicit dynamic transition tracking states.

The unmapped neural predictions in Fig. 8 demonstrate excellent identification of the dynamic transition states. This high precision makes it possible to reconstruct the continuous underlying states using sequential heuristic logic.

The tracking loop processes these outputs using active locomotion behaviors (walking, running, stairs_up, stairs_down) as stable anchor states to reset and calibrate the tracking baseline. If an upright_still sequence occurs immediately after a dynamic anchor class without a transition trigger, it is classified as standing. If a valid transition_down event is identified right after locomotion, the downstream upright_still segments are classified as sitting.

Once post-processing is complete, the helper transition categories are removed, leaving only the primary, full-duration human activities.

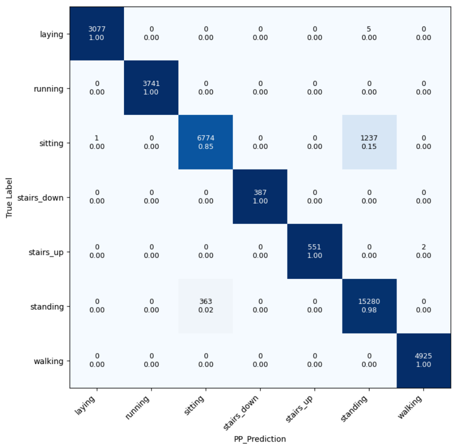

Fig. 9. Final post-processed confusion matrix resolving sitting and standing via state tracking.

TABLE I
DETAILED PERFORMANCE REPORT AFTER APPLYING STATE-TRANSITION TRACKING LOGIC.

Activity Class	**Precision**	**Recall**	**F_1-Score**	**Support**
Laying	0.9997	0.9984	0.9990	3,082
Running	1.0000	1.0000	1.0000	3,741
Sitting	0.9491	0.8455	0.8943	8,012
Stairs Down	1.0000	1.0000	1.0000	387
Stairs Up	1.0000	0.9964	0.9982	553
Standing	0.9248	0.9768	0.9501	15,643
Walking	0.9996	1.0000	0.9998	4,925
Accuracy	0.9558	0.9558	0.9558	36,343
Macro Average	**0.9819**	**0.9739**	**0.9773**	36,343
Weighted Average	0.9563	0.9558	0.9551	36,343

The final tracking metrics are summarized in Table I and Fig. 9. This tracking approach increases the sitting class F_1-score to 0.8943 and brings standing up to 0.9501, achieving a final pipeline macro F_1-score of 0.9773.

The remaining performance bottleneck is concentrated in sitting recall (0.8455). This issue stems from instances where the network misses brief, subtle `transition_down` events, causing the tracking loop to miss the change and misclassify sitting intervals as continuous standing states.

V. FUTURE WORK AND DIRECTIONS

To address the limitations identified during empirical evaluation and build upon the state-transition tracking framework, several key research avenues should be pursued.

A. Dataset Enrichment and Transition Refinement

The primary bottleneck in the current state-transition framework stems from sitting recall drops caused by false-negative errors in the `transition_down` class. To prevent continuous segments of motionless postures from being misclassified due to a single missed event trigger, future work needs to prioritize data enrichment strategies. This will involve applying time-series data augmentation techniques specifically targeting the transient temporal features of physical boundaries.

Furthermore, future dataset recording protocols should be restructured. Although the current experimental protocol featured a dedicated, intensive transition segment, it proved insufficient for capturing realistic state switches. Concentrating multiple transitions into a short timeframe introduces localized sensor drift. Future collection campaigns should maybe abandon isolated transition blocks, opting instead to naturally scatter a higher frequency of postural movements throughout continuous everyday activity protocols.

B. Statistical Validation of ABSMAX Significance

While the absolute maximum pooling layer (`AbsMaxPool1d`) demonstrated a consistent mathematical edge over standard Max-Pooling across both primary and external validation datasets, the absolute margins appeared in the third and fourth decimal places. Future research might focus on performing comprehensive statistical significance testing (such as McNemar's test [4] or paired t-tests on cross-validation folds) to formally validate whether preserving bidirectional zero-centered physical signal extremes offers a statistically robust improvement across a wider range of hardware sampling rates.

C. Scalability and Broad-Spectrum Validation

To further verify the pipeline's real-world generalization capabilities, more extensive testing needs to be conducted. This includes deploying the framework against a broader variety of external benchmark data configurations, evaluating its performance across diverse user demographic groups (e.g., varying body weights, heights, and age brackets), and analyzing the model's sensitivity to alternative on-body sensor placements.

D. Metabolic Tracking and Health Metrics Integration

Once the underlying classification architecture achieves near-perfect segmentation metrics, the pipeline will expand beyond core activity recognition. The robustly isolated behavioral segments will be used as a foundation for metabolic health metrics, specifically estimating active burned calories. By mapping granular states (including precise stair locomotion intensities and static postural durations) to metabolic equivalents (METs) and fusing these profiles with underlying sensor signal characteristics, the framework can be extended into a highly accurate health-monitoring system. [5]

REFERENCES

[1] H. Gjoreski, S. Kozina, M. Lustrek, and M. Gams, "Using multiple contexts to distinguish standing from sitting with a single accelerometer," vol. 263, 2014, p. . doi: 10.3233/978-1-61499-419-0-1015.
[2] M. Marinova *et al.*, "Deep Learning for Facial Expression and Human Activity Recognition Using Smart Glasses," *IEEE Access*, vol. 13, p. 3551610, 2025, doi: 10.1109/ACCESS.2025.3551610.
[3] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://openreview.net/forum?id=Skq89Scxx

[4] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.

[5] S. D. Herrmann *et al.*, “2024 Adult Compendium of Physical Activities: A third update of the energy costs of human activities,” *Journal of Sport and Health Science*, vol. 13, no. 1, pp. 6–12, 2024, doi: 10.1016/j.jshs.2023.10.010.